Instructor Handout 8: CPU Scheduling

Lecture 8 — Concepts, Criteria, FCFS and SJF

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

8.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 5: §5.1 basic concepts (p. 183: burst cycle, CPU scheduler, preemption, dispatcher), §5.2 criteria (p. 187), §5.3.1 FCFS (p. 188), §5.3.2 SJF and SRTF (p. 189). Primary anchor. *Page-verified* via the course index.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 9 §9.1 types of scheduling (p. 406) and §9.2 scheduling algorithms (p. 410). Stallings names the policies FCFS, SPN (non-preemptive SJF) and SRT; warn students who read both books. *Page-verified.*
- **Fix your conventions before class and announce them.** Ties go to the earlier arrival; SRTF preempts only on a *strictly* shorter remaining time; context-switch cost is ignored unless stated. Every instance in the deck and in this handout is tie-free, so the conventions never change an answer — but students will ask.
- **Prerequisites (two minutes).** Lecture 6: the ready queue, the short-term scheduler, and the context switch. Students should be able to say what is in a PCB.
- **Bring graph-ruled paper or draw a ruled time axis.** Every Gantt chart in this lecture uses integer milliseconds; a pre-drawn axis 0–30 saves several minutes.

Teaching note: The thesis for the hour: the policy does not change the work, only the order — and the order changes the waiting. Every Gantt chart today finishes at the same time under every policy; say so each time.

8.2 Opening: Five Jobs, One CPU

Five compile jobs arrive at the lab server's single CPU within five milliseconds (Table 8.1). Ask: in what order should they run, and how would you decide one order is better? We will schedule these jobs three ways — FCFS, SJF, SRTF — and measure each schedule with the same yardsticks. The

Process	P1	P2	P3	P4	P5
Arrival time (ms)	0	1	2	3	5
CPU burst τ (ms)	6	8	2	5	4

Table 8.1: The lab-server workload — keep it on a side board all hour.

roadmap is four questions: why schedule at all; what makes a schedule good; what the simplest policy does; and whether we can do better.

Board: copy Table 8.1 onto a side panel you will not erase. *On the board, in order:* (i) the process row; (ii) the arrival row; (iii) the burst row; (iv) below it, three empty lines labelled FCFS, SJF, SRTF with "avg WT = ?". *Say:* "By the end of the hour these three blanks will hold 9.2, 6.4, and 6.0."

> **Teaching note:** Before any definitions, ask for a show of hands: "Who would run `P3` first?" Many will, even though it has not arrived at time 0. That instinct — "short first" — is SJF, and the objection "it isn't there yet" is exactly the difference between SJF and SRTF. Note both and return to them.

8.3 Basic Concepts

The burst cycle

A process alternates between a **CPU burst** and an **I/O burst**, starting and ending with a CPU burst. Measured burst lengths are dominated by many short bursts and a few long ones. An **I/O-bound** program has many short CPU bursts; a **CPU-bound** program has a few long ones. Scheduling is worthwhile precisely because of the I/O bursts: while one process waits, another can compute.

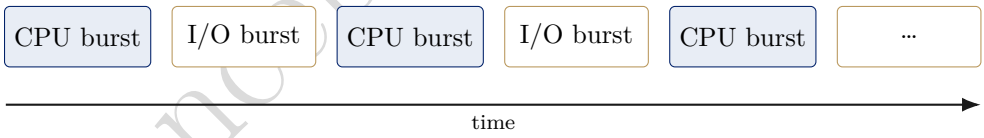


Figure 8.1: The CPU–I/O burst cycle (Silberschatz et al., Ch. 5 §5.1, p. 183).

How to present Figure 8.1. Follow the coordinate map in the source: draw the time arrow first, then the boxes left to right, alternating colours (CPU filled, I/O outlined). Say "compute, wait, compute, wait" as you draw. End with the ellipsis box and ask what the last box of any process must be (a CPU burst ending in `exit`).

The scheduler, preemption, the dispatcher

Definition (CPU scheduler). *The short-term scheduler selects a ready process in memory and allocates the CPU to it.*

The ready queue holds PCBs and is not necessarily FIFO (FIFO, priority queue, tree, or unordered list); the *policy* is the rule that picks. A decision may be needed when a process (1) goes running → waiting, (2) running → ready, (3) waiting → ready, (4) terminates. **Non-preemptive**

Concepts That Click

scheduling decides only at 1 and 4; **preemptive** scheduling also at 2 and 3, and must then protect shared data from half-finished updates.

Definition (Dispatcher and dispatch latency). *The dispatcher gives the CPU to the selected process: it switches context, switches to user mode, and jumps to the saved PC. The time to stop one process and start another is the dispatch latency.*

Board: the four transitions on Lecture 6's state diagram. *On the board, in order:* (i) draw ready, running, waiting as three circles; (ii) draw the four arrows and number them 1–4 (running→waiting, running→ready, waiting→ready, running→terminated); (iii) circle 1 and 4 in one colour and write "must decide"; (iv) circle 2 and 3 in another and write "may decide (preemptive only)". *Say:* "In 1 and 4 the CPU is empty; in 2 and 3 someone is still sitting in it."

Board: trace a disk read. *On the board, in order:* (i) a process issues `read` — ask which arrow (1); (ii) ask whether a non-preemptive scheduler must decide (yes: the CPU is free); (iii) the read completes — which arrow (3); (iv) ask again (no: the running process keeps the CPU; only a preemptive scheduler may reconsider). This is the deck's first Socratic check.

> **Teaching note:** Students confuse "preemptive" with "has a timer". Point at arrow 3: SRTF preempts on an *arrival*, with no timer involved. The timer belongs to Round Robin next lecture.

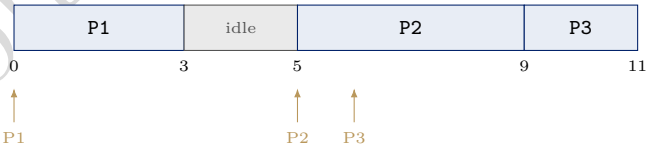
8.4 Scheduling Criteria

The five criteria: CPU utilization and throughput (maximize); turnaround time (TAT), waiting time (WT) and response time (RT) (minimize). The policy directly controls only WT; we usually optimize the average, while interactive systems also care about the variance of RT.

Definition (TAT, WT, RT). $\mathrm{TAT}_i = \mathrm{CT}_i - \mathrm{AT}_i, \quad \mathrm{WT}_i = \mathrm{TAT}_i - \tau_i, \quad \mathrm{RT}_i = \mathrm{ST}_i - \mathrm{AT}_i.$

WT counts all time ready but not running, including after a preemption; for a non-preemptive policy $\mathrm{WT}_i = \mathrm{RT}_i$; idle CPU time is nobody's waiting time.

Board: the idle-gap example from the deck. Jobs `P1` $(0, 3)$, `P2` $(5, 4)$, `P3` $(6, 2)$, FCFS. *On the board, in order:* (i) the axis 0–11 and arrival arrows at 0, 5, 6 below it; (ii) the bars `P1` 0–3, grey idle 3–5, `P2` 5–9, `P3` 9–11; (iii) the table, filling CT first, then TAT, then WT; (iv) utilization $9/11 \approx 81.8\%$.



Proc.	AT	τ	CT	TAT	WT	RT
`P1`	0	3	3	3	0	0
`P2`	5	4	9	4	0	0
`P3`	6	2	11	5	3	3
Average				4	1	1

Board: dispatch latency on the lab server. *On the board, in order:* (i) total burst $6+8+2+5+4 = 25$; (ii) FCFS makes 5 dispatches at 0.5 ms: $25 + 2.5 = 27.5$; (iii) utilization $25/27.5 \approx 90.9\%$, throughput $5/27.5 \approx 0.18$ jobs/ms; (iv) next to it, the zero-latency figures 100% and 0.2. *Say:* "Every switch is CPU time no job gets."

Concepts That Click

Teaching note: The commonest error of the whole lecture is computing WT as CT − AT. Make the class say the chain aloud: "completion minus arrival is turnaround; turnaround minus burst is waiting." Live question (Socratic check 2): for a laptop with an editor, a browser and a video export, which criterion first? Response time — maximizing throughput might freeze the editor behind the export.

8.5 First-Come, First-Served

Definition (FCFS). *The CPU goes to the process that requested it first; the ready queue is FIFO (join at the tail, take from the head). FCFS is non-preemptive.*

FCFS is a linked list with head and tail pointers. Its average WT is often long and depends heavily on arrival order. The **convoy effect**: one CPU-bound process holds the CPU while I/O-bound processes queue behind it with the devices idle; then they all block on I/O and the CPU idles; everyone moves at the speed of the slowest. FCFS is unsuitable for time-sharing (Silberschatz et al., Ch. 5 §5.3.1, pp. 188–189).

Board: the long job first (deck). Bursts 24, 3, 3 at time 0, order `P1`, `P2`, `P3`. *On the board, in order:* (i) a long `P1` bar 0–24, deliberately drawn long; (ii) two short bars to 27 and 30; (iii) WT $0, 24, 27$; (iv) average 17.



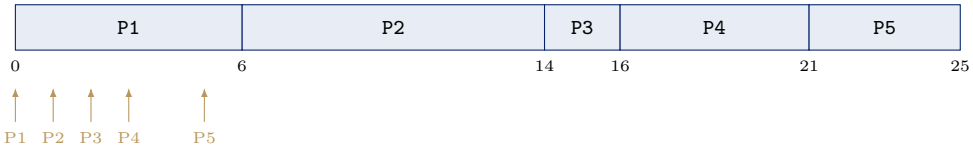
Prediction trap: before redrawing, ask the class to guess the average if the two short jobs had arrived first. Then draw `P2` 0–3, `P3` 3–6, `P1` 6–30: WT $6, 0, 3$, average 3. Same work, same finish time, $17/3 \approx 5.7$ times less waiting.



Board: a second instance for practice (not in the deck). Bursts 5, 2, 1 at time 0 in that order. FCFS: `P1` 0–5, `P2` 5–7, `P3` 7–8; WT $0, 5, 7$; average $12/3 = 4$. Keep this on the board: you will reuse it under SJF.



Board: the lab server under FCFS. *On the board, in order:* (i) bars in arrival order: `P1` 0–6, `P2` 6–14, `P3` 14–16, `P4` 16–21, `P5` 21–25; (ii) arrival arrows 0, 1, 2, 3, 5; (iii) the table row by row; (iv) point at `P3`: needs 2, waits 12. Average WT $46/5 = 9.2$ — fill the first blank on the side board.



Concepts That Click

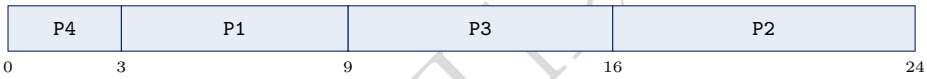
Proc.	AT	τ	CT	TAT	WT	RT
P1	0	6	6	6	0	0
P2	1	8	14	13	5	5
P3	2	2	16	14	12	12
P4	3	5	21	18	13	13
P5	5	4	25	20	16	16
Average				14.2	9.2	9.2

Teaching note: Students sometimes start P2 at its arrival time, overlapping P1. Insist: a bar starts at max(arrival, end of previous bar). The idle-gap example is the only place a bar starts at its arrival after a gap.

8.6 Shortest-Job-First

Definition (SJF). *The CPU goes to the ready process with the smallest next CPU burst; ties FCFS. Non-preemptive in this form (Stallings: SPN).*

Board: SJF with simultaneous arrivals (deck). Bursts 6, 8, 7, 3 at 0. *On the board, in order:* (i) sort the bursts: 3, 6, 7, 8; (ii) bars P4 0–3, P1 3–9, P3 9–16, P2 16–24; (iii) WT $3, 16, 9, 0$, average 7; (iv) beside it FCFS: $(0 + 6 + 14 + 21)/4 = 10.25$.



Board: return to the 5, 2, 1 instance. SJF: P3 0–1, P2 1–3, P1 3–8; WT $3, 1, 0$, average $4/3 \approx 1.33$ against FCFS's 4.

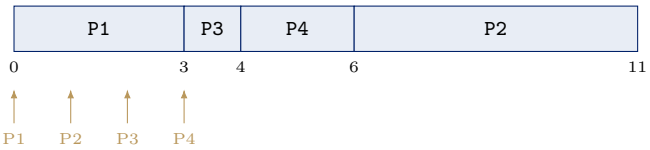


Why SJF is optimal

With n processes ready at 0, the k-th burst delays the $n - k$ after it: $\sum_i \mathrm{WT}_i = \sum_k (n-k)\tau_{(k)}$. The biggest weight goes with the first burst, so put the smallest burst there. Swapping an adjacent long–short pair (L, S) to short–long changes the sum by $S - L < 0$. SJF gives the minimum average WT (Silberschatz et al., Ch. 5 §5.3.2, from p. 189).

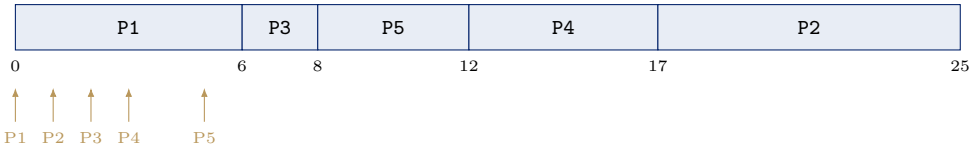
Board: the weights on the 5, 2, 1 instance. *On the board, in order:* (i) write the weights $2, 1, 0$ over three slots; (ii) put $5, 2, 1$ under them: $10 + 2 + 0 = 12$; (iii) put $1, 2, 5$: $2 + 2 + 0 = 4$; (iv) divide by 3: 4 and 1.33, exactly the two averages already on the board.

Board: SJF with arrivals (not in the deck). (AT, τ): P1 $(0, 3)$, P2 $(1, 5)$, P3 $(2, 1)$, P4 $(3, 2)$. Only P1 is ready at 0, so it runs 0–3; at 3 the choice is $\{5, 1, 2\}$: P3 3–4, P4 4–6, P2 6–11. WT $0, 5, 1, 1$, average 1.75 (FCFS gives 3.5).



Concepts That Click

Board: the lab server under SJF. P1 0–6 (alone at 0); at 6 the choice is $\{8, 2, 5, 4\}$: P3 6–8, P5 8–12, P4 12–17, P2 17–25. Average WT $32/5 = 6.4$ — fill the second blank.



Proc.	AT	τ	CT	TAT	WT	RT
P1	0	6	6	6	0	0
P2	1	8	25	24	16	16
P3	2	2	8	6	4	4
P4	3	5	17	14	9	9
P5	5	4	12	7	3	3
Average				11.4	6.4	6.4

Teaching note: Where students get stuck: at $t = 0$ they pick P3 because it is shortest overall. Ask "is P3 in the ready queue at time 0?" SJF chooses only among processes that have arrived.

Predicting the next burst

The scheduler cannot know the next burst; it predicts it by the exponential average

$$\hat{\tau}_{n+1} = \alpha\, \tau_n + (1 - \alpha)\, \hat{\tau}_n, \qquad 0 \le \alpha \le 1,$$

where τ_n is the measured n-th burst. $\alpha = 0$ ignores recent behaviour; $\alpha = 1$ uses only the last burst; the burst j steps back has weight $\alpha(1-\alpha)^j$. (Silberschatz writes t_n for the measured burst.)

Board: the deck's instance, $\alpha = 1/2$, $\hat{\tau}_0 = 10$, bursts $6, 4, 6, 4, 13, 13, 13$. *On the board, in order:* (i) two rows, "measured" and "predicted"; (ii) fill predicted one cell at a time, each the average of the cell to its upper left and the cell to its left: $10, 8, 6, 6, 5, 9, 11, 12$; (iii) circle the lag after the jump to 13.

Board: a second instance, $\alpha = 0.25$, $\hat{\tau}_0 = 8$, bursts 4, 12, 6: $\hat{\tau}_1 = 1{+}6 = 7$, $\hat{\tau}_2 = 3{+}5.25 = 8.25$, $\hat{\tau}_3 = 1.5 + 6.1875 = 7.6875$. Ask the class to predict whether it will track the jump to 12 faster or slower than the $\alpha = 1/2$ case (slower).

Teaching note: Live question (Socratic check 3): a process alternates 2-ms and 20-ms bursts — α near 0 or near 1? Near 1 is always wrong (predicts 2 before a 20); near 0 settles near 11. SJF is only as good as its predictor.

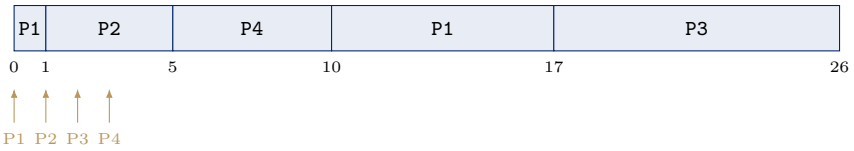
Concepts That Click

8.7 Shortest-Remaining-Time-First

Definition (SRTF). *Preemptive SJF: on each arrival, compare the newcomer's burst with the running process's remaining time; preempt if strictly shorter (Stallings: SRT).*

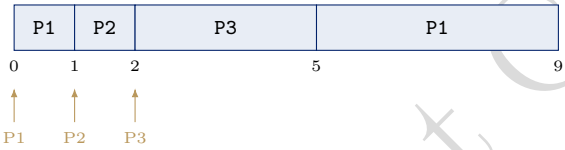
Decisions happen at every arrival and completion; equal remaining times do not preempt; WT includes time spent back in the queue after a preemption.

Board: the deck's SRTF instance, P1 $(0, 8)$, P2 $(1, 4)$, P3 $(2, 9)$, P4 $(3, 5)$. *On the board, in order:* (i) a "remaining" column you update at each event; (ii) at 1: 4 vs 7 — preempt; (iii) at 2 and 3: no preemption; (iv) at 5: P4 (5); at 10: P1 (7) before P3 (9); (v) the table. Average WT 6.5, against 7.75 non-preemptively. Point at P1: RT 0 but WT 9.

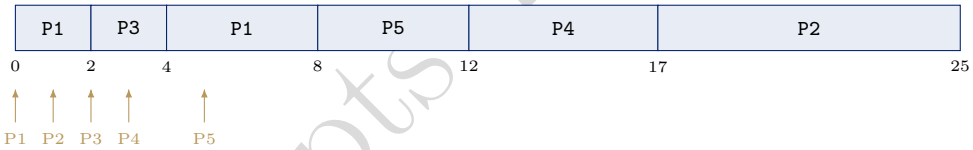


Proc.	AT	τ	CT	TAT	WT	RT
P1	0	8	17	17	9	0
P2	1	4	5	4	0	0
P3	2	9	26	24	15	15
P4	3	5	10	7	2	2
Average				13	6.5	4.25

Board: a second, small instance (not in the deck). P1 $(0, 5)$, P2 $(1, 1)$, P3 $(2, 3)$. At 1, P2 (1) beats 4 and runs 1–2; at 2, P3 (3) beats 4 and runs 2–5; P1 5–9. WT $4, 0, 0$, average 1.33; non-preemptive SJF gives 2.67. Four dispatches.



Board: the lab server under SRTF. At 2, P3 (2) beats P1's 4; at 3 and 5 the newcomers (5 and 4) do not beat what is running (1 and 3 left). Average WT $30/5 = 6.0$, average RT 5.6 — fill the last blank.



Proc.	AT	τ	CT	TAT	WT	RT
P1	0	6	8	8	2	0
P2	1	8	25	24	16	16
P3	2	2	4	2	0	0
P4	3	5	17	14	9	9
P5	5	4	12	7	3	3
Average				11	6	5.6

Teaching note: Prediction trap: before the lab-server SRTF trace, ask "how many preemptions?" Most guess three or four (one per arrival). The answer is one. Arrivals only *offer* a preemption; the comparison decides.

8.8 Comparing the Policies; Starvation

Policy	avg WT	avg TAT	avg RT	dispatches
FCFS	9.2	14.2	9.2	5
SJF (non-preemptive)	6.4	11.4	6.4	5
SRTF	6.0	11.0	5.6	6

Concepts That Click

All three finish at 25 ms. SRTF buys the best averages with one extra dispatch; at 0.5 ms latency its utilization is $25/28 \approx 89.3\%$ against 90.9% for FCFS. SJF and SRTF can **starve** long jobs: `P2` already has the largest WT (16) under both, and a steady stream of 2-ms jobs would postpone it forever. FCFS never starves. Further costs: SJF/SRTF need a prediction; SRTF pays a dispatch per preemption. Ageing (Lecture 9) cures starvation.

Board: build starvation live. Leave `P2` $(1, 8)$ waiting and add a 2-ms job arriving every 2 ms from $t = 4$; ask the class to extend the SRTF chart until someone notices `P2` never gets the CPU.

> **Teaching note:** Close with the misconceptions slide read as a quiz: "WT is CT minus AT" (no, that is TAT); "SJF uses total job length" (next burst); "SRTF preempts on ties" (strictly shorter only); "a better policy finishes sooner" (same last completion, different averages).

8.9 Exit Ticket / Comprehension Checks

1. A process issues a disk read; later the read completes. Which of these moments force a non-preemptive decision? *Answer:* only the read (running $\rightarrow$ waiting).
2. Which criterion matters first on an interactive laptop? *Answer:* response time.
3. Bursts 5, 2, 1 at 0: FCFS and SJF average WT? *Answer:* 4 and 1.33.
4. $\alpha = 1/2$, $\hat{\tau}_0 = 10$, first burst 6: $\hat{\tau}_1$? *Answer:* 8.
5. How many preemptions does SRTF make on the lab-server jobs? *Answer:* one (at $t = 2$).

8.10 Summary: One-Page Refresh

- CPU and I/O bursts alternate; the short-term scheduler fills the CPU during I/O waits.
- Decisions on block/exit (non-preemptive) and also on timer/I/O completion (preemptive); the dispatcher pays a latency per switch.
- TAT = CT − AT, WT = TAT − τ, RT = ST − AT.
- FCFS: simple, starvation-free, convoy effect. Lab server: avg WT 9.2.
- SJF: optimal average WT, needs exponential-average prediction. Lab server: 6.4.
- SRTF: preemptive SJF, strictly-shorter rule. Lab server: 6.0, one extra dispatch. Both SJF and SRTF can starve long jobs.

Concepts That Click